

Deployment Guide

JEPA Event-Option Live Stack

Kripta Studios

July 2026

Contents

1	Scope	2
2	Pre-Deployment Checks	2
3	Deploy on VPS	2
4	Expected Service Commands	2
5	Expected Logs	3
6	Daily Audit	3
7	State Reset	4
8	Retraining Cadence	4

1 Scope

This guide deploys the current live event-option package. It does not deploy the old GBT+RL bot, Hybrid Attention-MLP bot, OptionValue selector, or 180 minute JEPA direction runner.

2 Pre-Deployment Checks

Compile the live files:

```
python3 -m py_compile \  
  services/realtime_feed.py \  
  bots/tradingbot_wrapper_jepa.py \  
  neural/jepa/event_option_live_snapshot.py \  
  neural/jepa/event_option_live_scorer.py \  
  neural/jepa/event_option_component_live.py \  
  neural/jepa/audit_live_feature_drift.py \  
  neural/jepa/validate_event_option_production_package.py
```

Validate the package:

```
python3 neural/jepa/validate_event_option_production_package.py \  
  --policy neural/models/jepa/  
    jepa_production_event_options_frozen2025_static_union_202607/event_option_policy  
    .json \  
  --registry neural/models/jepa/  
    jepa_production_event_options_frozen2025_static_union_202607/component_registry.  
    json \  
  --require-live-ready \  
  --ignore-raw-thetadate-coverage \  
  --min-profit-factor 1.3 \  
  --min-win-rate 0.45 \  
  --min-month-trades 12
```

3 Deploy on VPS

Use explicit service restarts. Do not use `reboot now` as the normal deployment mechanism.

```
cd /home/Option-Greeks-Plotting-Discord-Bot  
git pull --ff-only  
  
sudo install -m 0644 systemd/realtime_feed.service /etc/systemd/system/realtime_feed.  
  service  
sudo install -m 0644 systemd/ai_bot.service /etc/systemd/system/ai_bot.service  
sudo systemctl daemon-reload  
sudo systemctl enable realtime_feed.service ai_bot.service  
sudo systemctl restart realtime_feed.service  
sudo systemctl restart ai_bot.service  
sudo systemctl status realtime_feed.service ai_bot.service --no-pager -l
```

Use exactly `--no-pager -l`. Do not append a bare number such as 50; systemd interprets that as a PID.

4 Expected Service Commands

The feed unit should point to:

```

services/realtime_feed.py
--event-option-policy-path neural/models/jepa/
    jepa_production_event_options_frozen2025_static_union_202607/event_option_policy.
    json
--event-option-component-registry-path neural/models/jepa/
    jepa_production_event_options_frozen2025_static_union_202607/component_registry.
    json
--require-event-option-live-ready

```

The bot unit should point to:

```

bots/tradingbot_wrapper_jepa.py
--disable-level-signal
--disable-structural-profile
--disable-option-value
--require-event-option-policy
--require-event-option-component-registry
--require-event-option-live-ready
--enable-event-option-scorer
--strict-event-option-features
--paper-order-intents

```

5 Expected Logs

Feed:

```

[EVENT_OPTION] loaded policy=event_option_frozen2025_static_union_balanced_202607
[EVENT_OPTION] loaded component registry status=production_live_ready
Starting RealtimeOptionsFeed

```

Bot:

```

Loaded event-option production policy=
    event_option_frozen2025_static_union_balanced_202607
Loaded event-option component registry status=production_live_ready
Starting JEPA live bot ... event_exit=stop=-60%/tp=1000%/trail=50%/25%/min_hold=30m/
    max_hold=180m
Paper order intents enabled

```

6 Daily Audit

After the feed has produced a daily event-option snapshot:

```

DAY=$(TZ=America/New_York date +%Y%m%d)
python3 neural/jepa/audit_live_feature_drift.py --day-dir "rt_data/$DAY" --strict-
    features
journalctl -u realtime_feed.service -n 120 --no-pager
journalctl -u ai_bot.service -n 160 --no-pager
tail -n 200 trades_jepa/tradingbot_jepa.log

```

If `event_option_snapshots_latest.parquet` does not exist before market data arrives, the audit can fail with `FileNotFoundError`. That is not a production failure by itself.

7 State Reset

Before deleting bot state, verify there are no open positions:

```
python3 - <<'PY'
import json
from pathlib import Path
p = Path("trades_jepa/open_positions_jepa.json")
data = json.loads(p.read_text()) if p.exists() and p.read_text().strip() else {}
print(json.dumps(data, indent=2))
if data:
    raise SystemExit("Open positions exist; do not clear state")
PY
```

Only clear state intentionally and after archiving `trades_jepa/`.

8 Retraining Cadence

Do not retrain daily. The normal workflow is:

1. audit live behavior daily;
2. collect and verify completed-month data;
3. run leakage and walk-forward validation;
4. export a new production package only if it passes live-ready checks;
5. deploy with git pull, validation, service install, and restart.